TASK -10 - EBS SINGLE-ATTACH AND EFS SHARED STORAGE IN AWS

TITLE

Configure EBS with Single-Attach and Configure EFS for Sharing Between Multiple EC2 Instances

---

OBJECTIVE

GOAL: Configure an EBS such that it can be shared between different instances.

1. Create and set up an EBS.
2. Verify the working.
3. Study about Elastic File System (EFS) in AWS.
4. Configure an EFS shared between different EC2 instances.

PART 1 — EBS SINGLE-ATTACH

Step 1 — Launch EC2 Instance

An EC2 instance was launched in AWS for performing the EBS practical.

The EC2 instance was running **Amazon Linux 2023**.

The operating system was verified using:

cat /etc/os-release

The output confirmed that the instance was running Amazon Linux 2023.

Step 2 — Create an EBS Volume

A new EBS volume was created with the following configuration:

- Volume name: **new**
- Volume type: **gp3**
- Size: **10 GiB**
- Availability Zone: **ap-south-1b**
- Multi-Attach: **Disabled**

Since Multi-Attach was disabled, the volume was configured for **Single-Attach** behavior.

Single-Attach means that the EBS volume can be attached to one EC2 instance at a time.

Step 3 — Attach the EBS Volume

The newly created EBS volume was attached to the first EC2 instance.

The device name selected during attachment was:

/dev/sdf

After attaching the volume, it became available to the operating system.

Step 4 — Identify the EBS Device

The following command was used to identify the attached disks:

lsblk

The output showed:

```
nvme0n1       259:0    0   8G  0 disk
├─nvme0n1p1   259:2    0   8G  0 part /
├─nvme0n1p127 259:3    0   1M  0 part
└─nvme0n1p128 259:4    0  10M  0 part /boot/efi
nvme1n1       259:1    0  10G  0 disk
```

Here:

- /dev/nvme0n1 was the root EBS volume.
- /dev/nvme1n1 was the newly attached 10 GiB EBS volume.

The root volume was not modified.

Step 5 — Format the EBS Volume

The new EBS volume was formatted with the XFS filesystem:

sudo mkfs -t xfs /dev/nvme1n1

Formatting prepares the EBS volume so that it can be used to store files.

Step 6 — Create a Mount Point

A directory named /data was created to mount the EBS volume:

sudo mkdir -p /data

Step 7 — Mount the EBS Volume

The EBS volume was mounted to /data:

sudo mount /dev/nvme1n1 /data

The EBS volume was now accessible through:

/data

```
[ec2-user@ip-172-31-15-221 ~]$ sudo mkfs -t xfs /dev/nvme1n1
meta-data=/dev/nvme1n1           isize=512    agcount=16, agsize=163840 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
         =                       exchange=0
data     =                       bsize=4096   blocks=2621440, imaxpct=25
         =                       sunit=1      swidth=1 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1, parent=0
log      =internal log           bsize=4096   blocks=16384, version=2
         =                       sectsz=512   sunit=1 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
[ec2-user@ip-172-31-15-221 ~]$ sudo mkdir -p /data
[ec2-user@ip-172-31-15-221 ~]$ sudo mount /dev/nvme1n1 /data
[ec2-user@ip-172-31-15-221 ~]$ dh -Th
-bash: dh: command not found
[ec2-user@ip-172-31-15-221 ~]$ df -Th
Filesystem       Type      Size  Used Avail Use% Mounted on
devtmpfs         devtmpfs  4.0M     0  4.0M   0% /dev
tmpfs            tmpfs     457M     0  457M   0% /dev/shm
tmpfs            tmpfs     183M  444K  183M   1% /run
efivarfs         efivarfs  128K  3.0K  121K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p1   xfs       8.0G  1.7G  6.4G  21% /
tmpfs            tmpfs     457M     0  457M   0% /tmp
/dev/nvme0n1p128 vfat       10M  1.3M  8.7M  13% /boot/efi
tmpfs            tmpfs      92M     0   92M   0% /run/user/1000
/dev/nvme1n1     xfs        10G  104M  9.9G   2% /data
[ec2-user@ip-172-31-15-221 ~]$ lsblk
```

Step 8 — Create and Verify a File on the EBS Volume

After mounting the EBS volume, a test file was created inside /data.

The file was created using:

sudo touch file1

The file was opened using vi:

vi file1

The file was also opened with root privileges using:

sudo vi file1

The following text was entered into the file:

hellow this is text file

The file permissions and ownership were checked using:

sudo ls -l file1

The output showed:

-rw-r--r-- 1 root root 0 Sep 17 05:12 file1

Finally, the contents of the file were verified using:

cat file1

Output:

hellow this is text file

This confirmed that the EBS volume was successfully mounted and that a file could be created and accessed from the volume.

```
[ec2-user@ip-172-31-15-221 data]$ sudo cat >> test.txt
-bash: test.txt: Permission denied
[ec2-user@ip-172-31-15-221 data]$ sudo touch file1
[ec2-user@ip-172-31-15-221 data]$ vi file1
[ec2-user@ip-172-31-15-221 data]$ sudo ls -l file1
-rw-r--r--. 1 root root 0 Sep 17 05:12 file1
[ec2-user@ip-172-31-15-221 data]$ sudo vi file1
[ec2-user@ip-172-31-15-221 data]$ cat file1
hellow this is text file
[ec2-user@ip-172-31-15-221 data]$
```

Step 9 — Unmount and Detach the EBS Volume

After completing the test on the first EC2 instance, the EBS volume was unmounted.

First, the home directory was accessed:

cd ~

Then the EBS volume was unmounted:

sudo umount /data

After unmounting, the EBS volume was detached from the first EC2 instance through the AWS Management Console.

Step 10 — Identify and Mount the EBS Volume on the Second EC2

The attached volume was checked using:

lsblk

The 10 GiB EBS volume appeared as:

nvme1n1

A mount point was created:

sudo mkdir -p /data

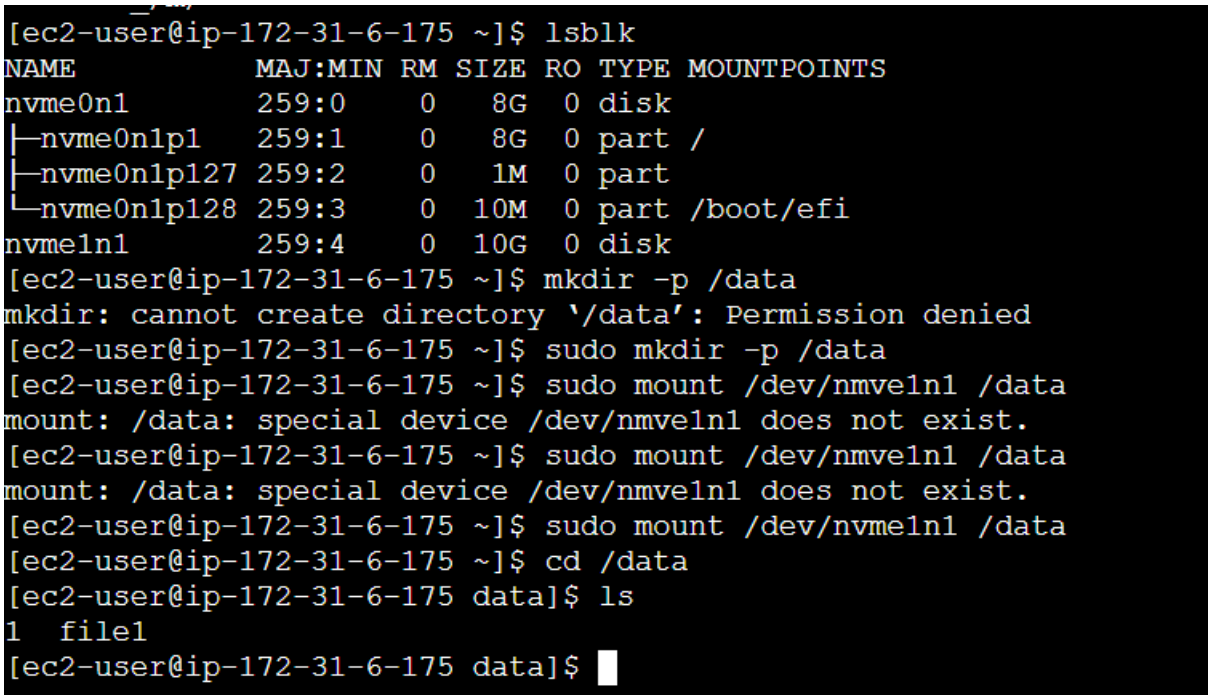
The EBS volume was then mounted:

sudo mount /dev/nvme1n1 /data

The previously created file was then accessed from the second EC2 instance.

Listed the files in that mounting point

This showed that the data stored on the EBS volume remained available after moving the volume from the first EC2 instance to the second EC2 instance.



EBS VERIFICATION

The EBS Single-Attach practical was successfully verified.

The following operations were completed:

- Created a 10 GiB gp3 EBS volume.
- Configured it with Multi-Attach disabled.
- Attached it to the first EC2 instance.
- Formatted it using XFS.
- Mounted it at /data.
- Created and verified file1.
- Unmounted and detached the EBS volume.
- Attached the same EBS volume to a second EC2 instance.
- Mounted the volume on the second instance.
- Verified that file1 and its contents were still available.

Note: Single-Attach EBS does not allow simultaneous access from multiple EC2 instances. The volume was moved from one instance to another to demonstrate data persistence.

PART 2 — ELASTIC FILE SYSTEM (EFS)

Step 1 — Study About EFS

Amazon Elastic File System (EFS) is a managed file storage service provided by AWS.

EFS provides a shared filesystem that can be accessed by multiple EC2 instances.

Unlike the Single-Attach EBS practical, the same EFS filesystem can be mounted by multiple EC2 instances at the same time.

EFS uses the **NFS protocol** for communication between EC2 instances and the filesystem.

Step 2 — Create an EFS File System

An EFS filesystem was created with the name:

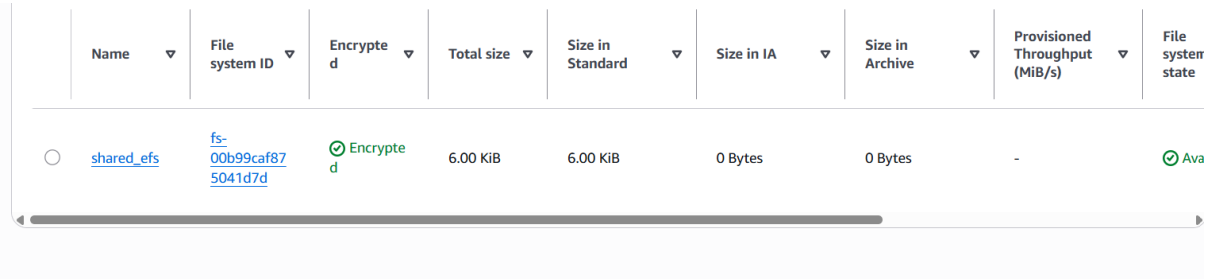
shared_efs

The EFS filesystem was created in the same VPC used by the EC2 instances.

The EFS filesystem ID was:

fs-0f4f759b6a1a309dc

The filesystem status was verified as **Available**.



Name	File system ID	Encrypted	Total size	Size in Standard	Size in IA	Size in Archive	Provisioned Throughput (MiB/s)	File system state
shared_efs	fs-00b99caf875041d7d	Encrypted	6.00 KiB	6.00 KiB	0 Bytes	0 Bytes	-	Ava

Step 3 — Configure EFS Mount Targets

EFS mount targets were configured in the required Availability Zones.

The mount targets were available in:

ap-south-1a
ap-south-1b
ap-south-1c

Mount targets allow EC2 instances inside the VPC to connect to the EFS filesystem.

Step 4 — Configure Security Group

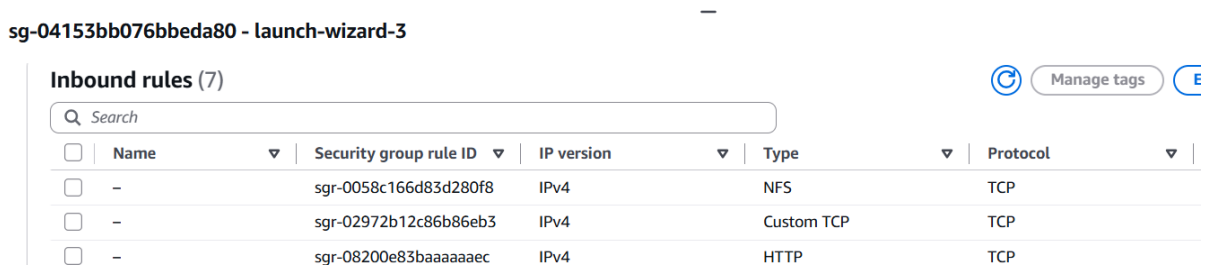
The EFS mount targets were configured to use the same security group as the EC2 instances.

An inbound security group rule was added for NFS:

Type: NFS
Protocol: TCP
Port: 2049
Source: EC2 Security Group

Port **2049** is used for NFS communication.

This rule allows the EC2 instances to communicate with the EFS mount targets.



Step 5 — Connect to the First EC2 Instance

The first EC2 instance used for the EFS practical was:

ebs_1

The operating system was verified using:

cat /etc/os-release

The output confirmed that the instance was running:

Amazon Linux 2023

Step 6 — Install EFS Utilities

The EFS utilities package was installed using:

sudo dnf install -y amazon-efs-utils

These utilities provide tools that help mount and work with Amazon EFS.

```
[ec2-user@ip-172-31-15-221 ~]$ cat /etc/os-release
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:al2023"
PRETTY_NAME="Amazon Linux 2023.12.20260914"
ANSI_COLOR="0;33"
CPE_NAME="cpe:2.3:o:amazon:amazon_linux:2023"
HOME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
DOCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
SUPPORT_URL="https://aws.amazon.com/premiumsupport/"
BUG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
VENDOR_NAME="AWS"
VENDOR_URL="https://aws.amazon.com/"
SUPPORT_END="2029-06-30"
[ec2-user@ip-172-31-15-221 ~]$ sudo dnf install -y amazon-efs-utils
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
================================================================================
 Package                    Architecture          Version
================================================================================
Installing:
 amazon-efs-utils           x86_64                3.3.1-1.amzn2023
Installing dependencies:
 stunnel                    x86_64                5.58-1.amzn2023.0.2
```

Step 7 — Create an EFS Mount Directory

A directory named /efs was created:

sudo mkdir -p /efs

This directory was used as the mount point for the EFS filesystem.

Step 8 — Mount EFS on the First EC2 Instance

The EFS filesystem was mounted using the EFS mount target IP address with NFS version 4.1:

sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport 172.31.42.241:/ /efs

After successful mounting, the EFS filesystem was accessed through the /efs directory.

The EFS mount was verified by entering the mount directory:

cd /efs

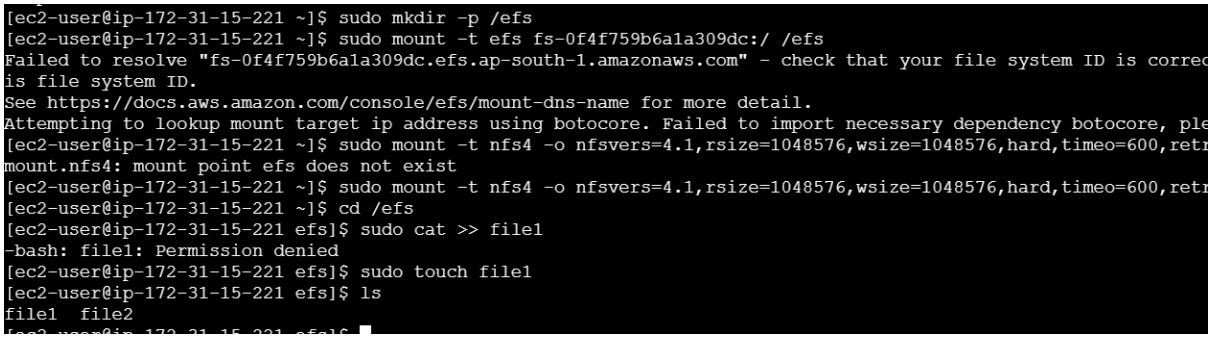
Step 9 — Create Test Files on EFS

After entering the EFS mount directory, test files were created to verify that the EFS filesystem was working correctly.

A file named file1 was created using:

sudo touch file1

This confirmed that files could be created and accessed inside the EFS-mounted directory.



Step 10 — Mount the Same EFS on the Second EC2 Instance

The second EC2 instance was connected to configure access to the same EFS filesystem.

A mount directory was created:

sudo mkdir -p /efs

The same EFS filesystem was mounted using the EFS mount target IP address:

sudo mount -t nfs4 -o
nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport
172.31.42.241:/ /efs

After successful mounting, the EFS filesystem was accessible through:

cd /efs

Step 11 — Verify Shared EFS Storage

The contents of the EFS directory were checked on the second EC2 instance:

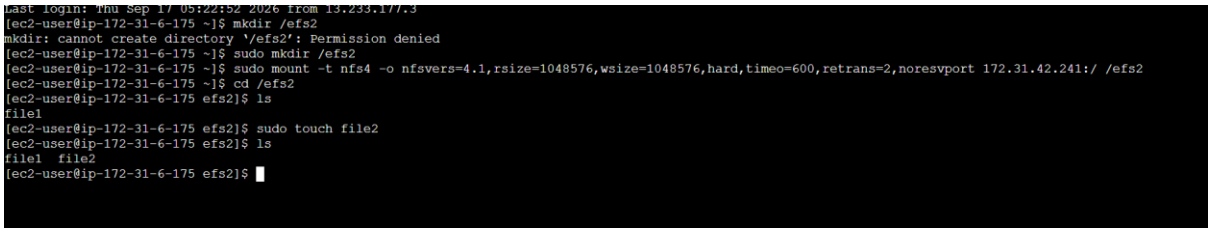
ls

The files created on the EFS filesystem were visible from the second EC2 instance.

The same EFS filesystem was therefore successfully accessed from both EC2 instances.

This verified that **EFS provides shared file storage that can be accessed by multiple EC2 instances**.

Added a file file2



EFS VERIFICATION

The EFS practical was successfully completed.

The following operations were verified:

- Created an EFS filesystem named shared_efs.
- Obtained EFS filesystem ID:

fs-0f4f759b6a1a309dc

- Configured EFS mount targets.
- Configured the security group for **NFS TCP port 2049**.
- Installed amazon-efs-utils.
- Created the /efs mount directory.
- Mounted the EFS filesystem using the mount target IP address:

172.31.42.241

- Accessed the EFS filesystem through /efs.
- Created test files on the EFS filesystem.
- Mounted the same EFS filesystem on the second EC2 instance.
- Verified that the EFS files were accessible from the second EC2 instance.

The successful access from both EC2 instances demonstrated **shared storage using Amazon EFS**

CONCLUSION

The practical successfully demonstrated the configuration and working of **EBS Single-Attach** and **Amazon EFS shared storage** in AWS.

For **EBS**, a 10 GiB gp3 volume with Multi-Attach disabled was created, attached to an EC2 instance, formatted, mounted, and used to store files. The volume was then detached and attached to another EC2 instance, where the previously stored data was successfully accessed. This verified data persistence when moving a Single-Attach EBS volume between EC2 instances.

For **EFS**, an EFS filesystem was created and mounted on multiple EC2 instances using NFS. Test files were created in the EFS-mounted directory and were accessible from the EC2 instances. This verified that EFS provides **shared file storage that can be accessed by multiple EC2 instances simultaneously**.